



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2025

Facultad de Ingeniería
UNLP

PERIFERICO TIMER
(AVR)

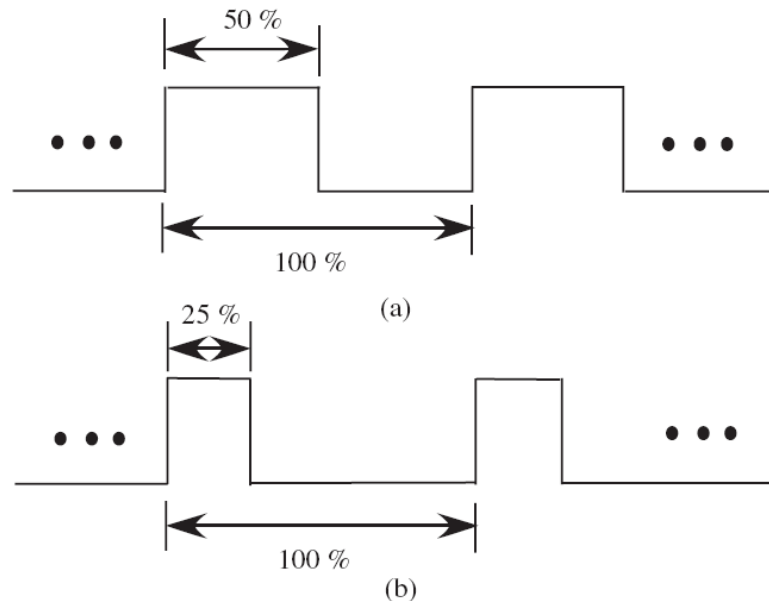
Ing. José Juárez

Introducción

- Una de las características mas destacables de un MCU es la capacidad de realizar tareas temporizadas.
- Para esto cuentan con un periférico TIMER o TEMPORIZADOR
- Algunas de las aplicaciones pueden ser:
 - Generación de retardos
 - Interrupción periódica de tiempo-real (planificación de tareas)
 - Protección Watch-Dog
- Pero además un Timer se puede utilizar para:
 - Generación de señales digitales con frecuencia variables o ciclo de trabajo variable (PWM)
 - Medición de frecuencia y ancho de pulso
 - Registro y conteo de eventos (COUNTER)

Periodo, Frecuencia y Ciclo de Trabajo

- El periodo T de una señal real $x(t)$, es el menor numero entero que satisface:
 $x(t)=x(t+T)$
- La frecuencia se define como el numero de oscilaciones en el lapso de 1 seg. Es decir: $f=1/T$
- El ciclo de trabajo de una señal digital es el porcentaje de tiempo en que la misma está activa respecto del periodo total: $(\text{ancho de pulso} * 100\%) / T$



Sistema básico de temporización

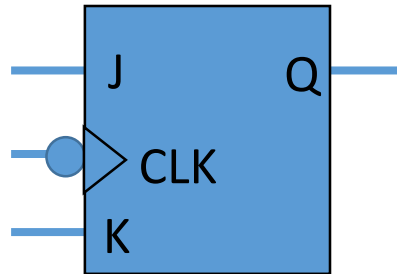
Un TIMER/COUNTER cuenta pulsos de reloj de una señal digital de CLK por lo tanto el tiempo es discreto (solo se puede ejecutar una acción en un flanco de la señal de CLK).

Vamos a definir algunos parámetros asociados a este problema y que vamos a utilizar después:

- **Resolución:** es el mínimo período de tiempo medible o contable y es 1 pulso de CLK o período de CLK (depende de la fuente de reloj y de la selección de los predivisores)
- **Rango:** es el rango de valores (Máx - Mín) que se utiliza para representar la información.
- **Precisión:** con cuantos bits puedo representar la información. Básicamente es el Número de bits del Timer.
- **Exactitud:** es cuanto difiere el “valor real” respecto al “valor medido”, depende de la exactitud del oscilador que genera la señal del reloj.
- **Estabilidad:** es una medida de cuan estable es la frecuencia del CLK frente a perturbaciones en la tensión de alimentación, en la temperatura y al envejecimiento de los componentes. Puede dividirse en estabilidad de corto término y estabilidad a largo plazo.

Sistema básico de temporización

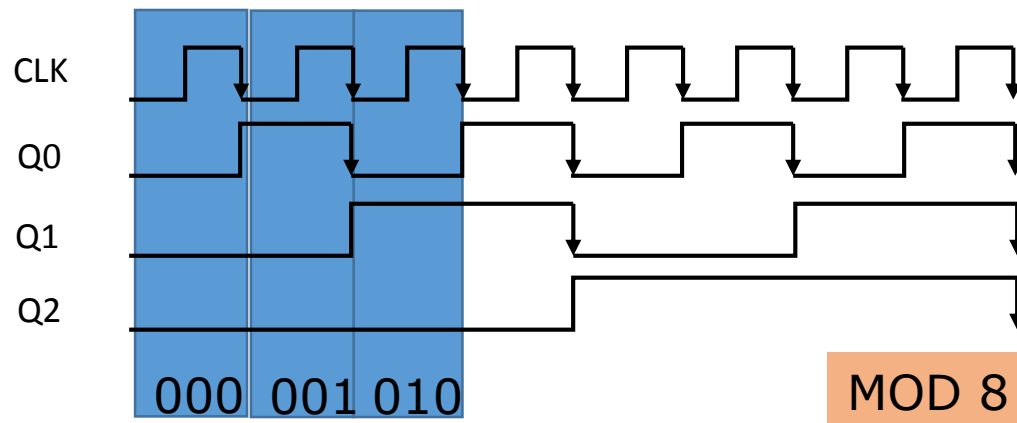
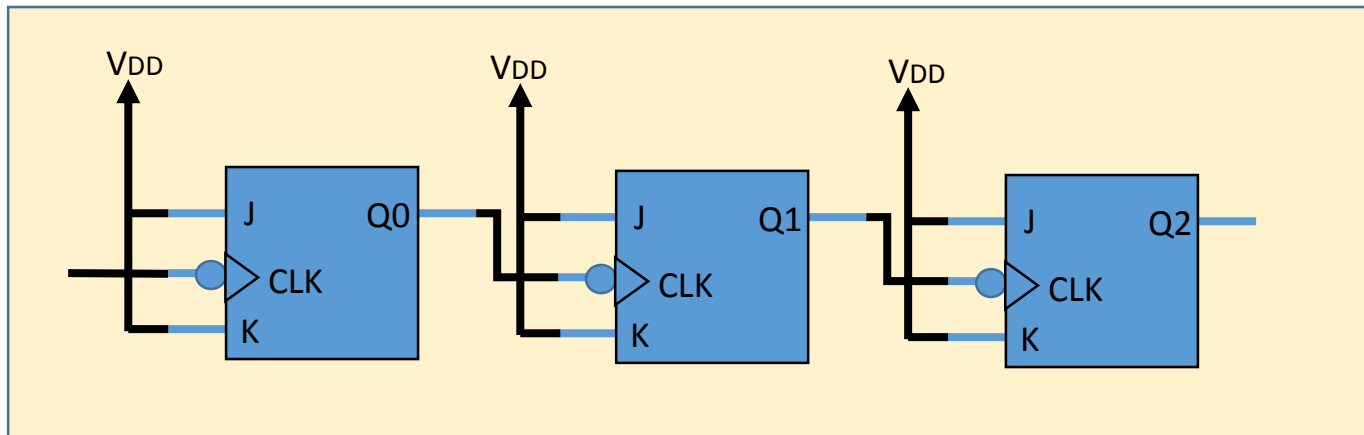
Flip flop J-K



J	K	CLK	Q
0	0	↓	Sin cambio
0	1	↓	0
1	0	↓	1
1	1	↓	toggle

Sistema básico de temporización

- Contador asincrónico de Rizo con 3 FF



f_{CLK}

$f_{CLK} / 2$

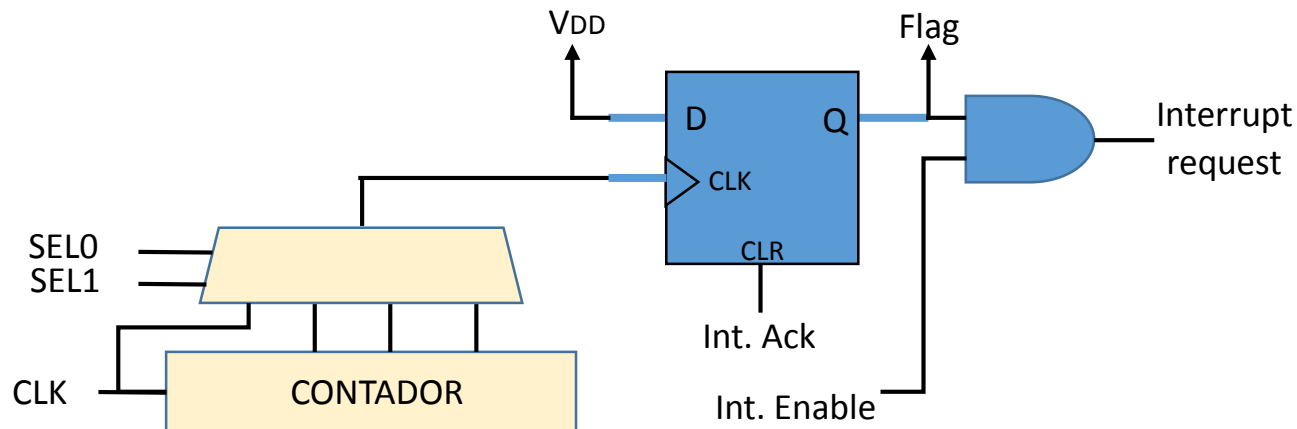
$f_{CLK} / 4$

$f_{CLK} / 8$

MOD 8

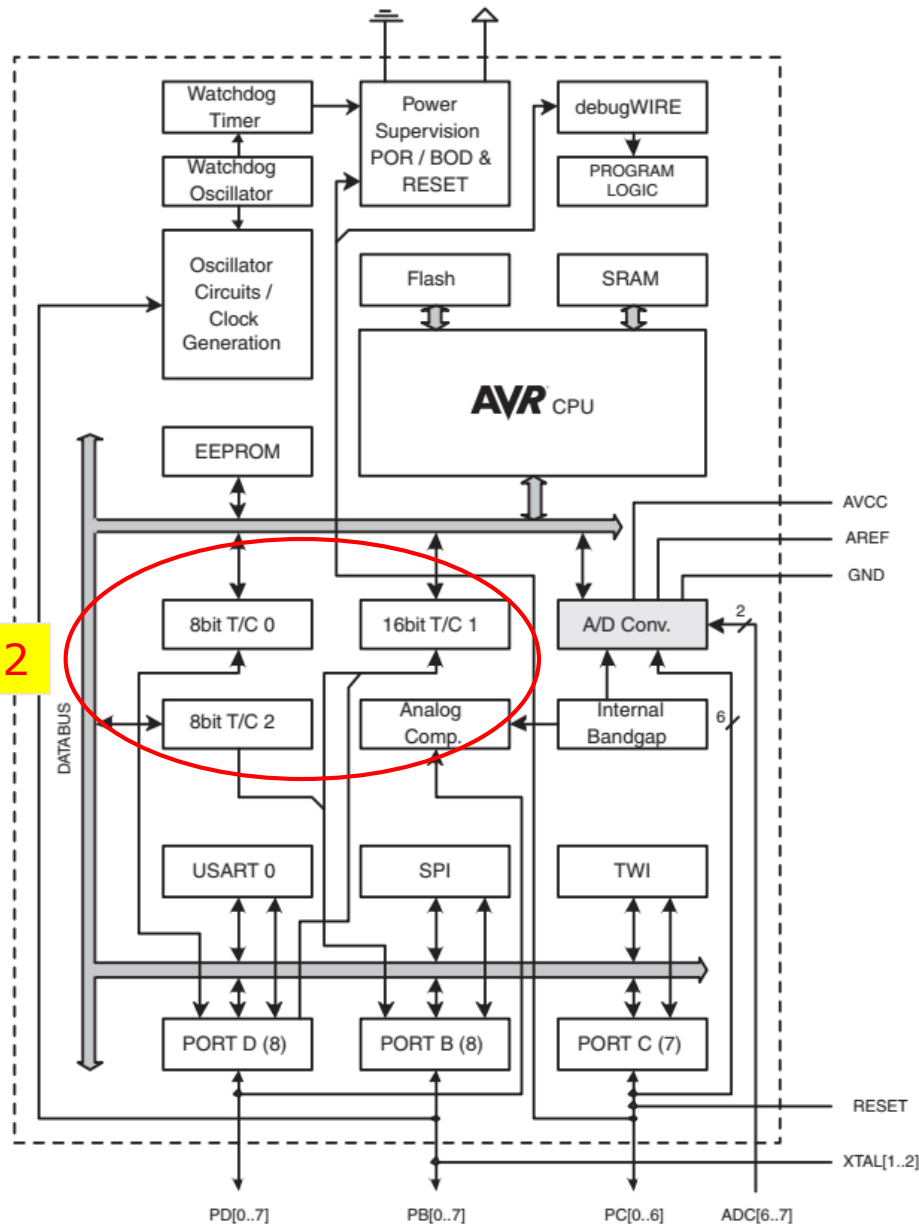
Sistema básico de temporización

- Generador de Interrupciones periódicas



SEL1	SEL0	Frecuencia de interrupción
0	0	$f_{CLK}/1$
0	1	$f_{CLK}/2$
1	0	$f_{CLK}/4$
1	1	$f_{CLK}/8$

Temporizadores en AVR ATmega328p



T/C: 0, 1 y 2



Temporizadores en AVR ATmega328p

Timer 0

- 8-bit timer/counter
- 10-bit clock prescaler
- Functions:
 - Pulse width modulation
 - Frequency generation
 - Event counter
 - Output compare
- Modes of operation:
 - Normal
 - Clear timer on compare match (CTC)
 - Fast PWM
 - Phase correct PWM

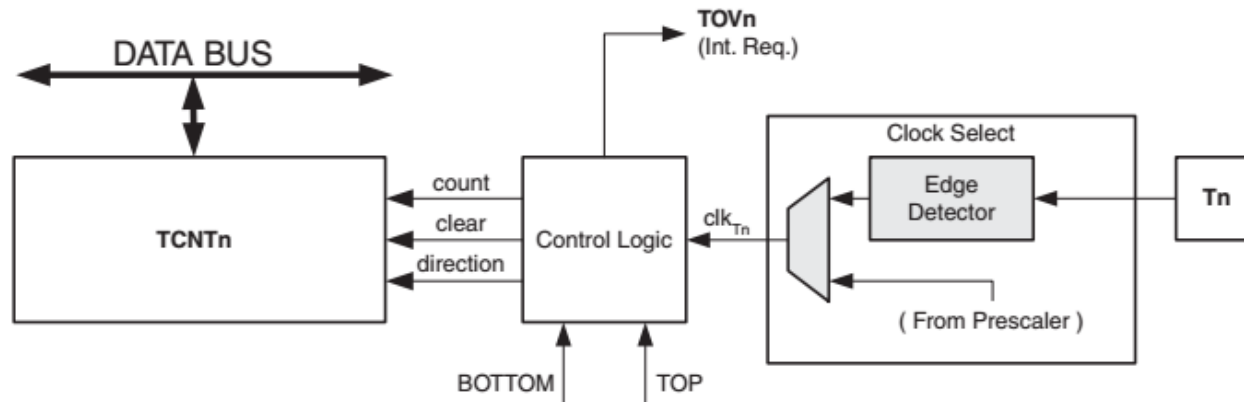
Timer 1

- 16-bit timer/counter
- 10-bit clock prescaler
- Functions:
 - Pulse width modulation
 - Frequency generation
 - Event counter
 - Output compare – 2 ch
 - Input capture
- Modes of operation:
 - Normal
 - Clear timer on compare match (CTC)
 - Fast PWM
 - Phase correct PWM

Timer 2

- 8-bit timer/counter
- 10-bit clock prescaler
- Functions:
 - Pulse width modulation
 - Frequency generation
 - Event counter
 - Output compare
- Modes of operation:
 - Normal
 - Clear timer on compare match (CTC)
 - Fast PWM
 - Phase correct PWM

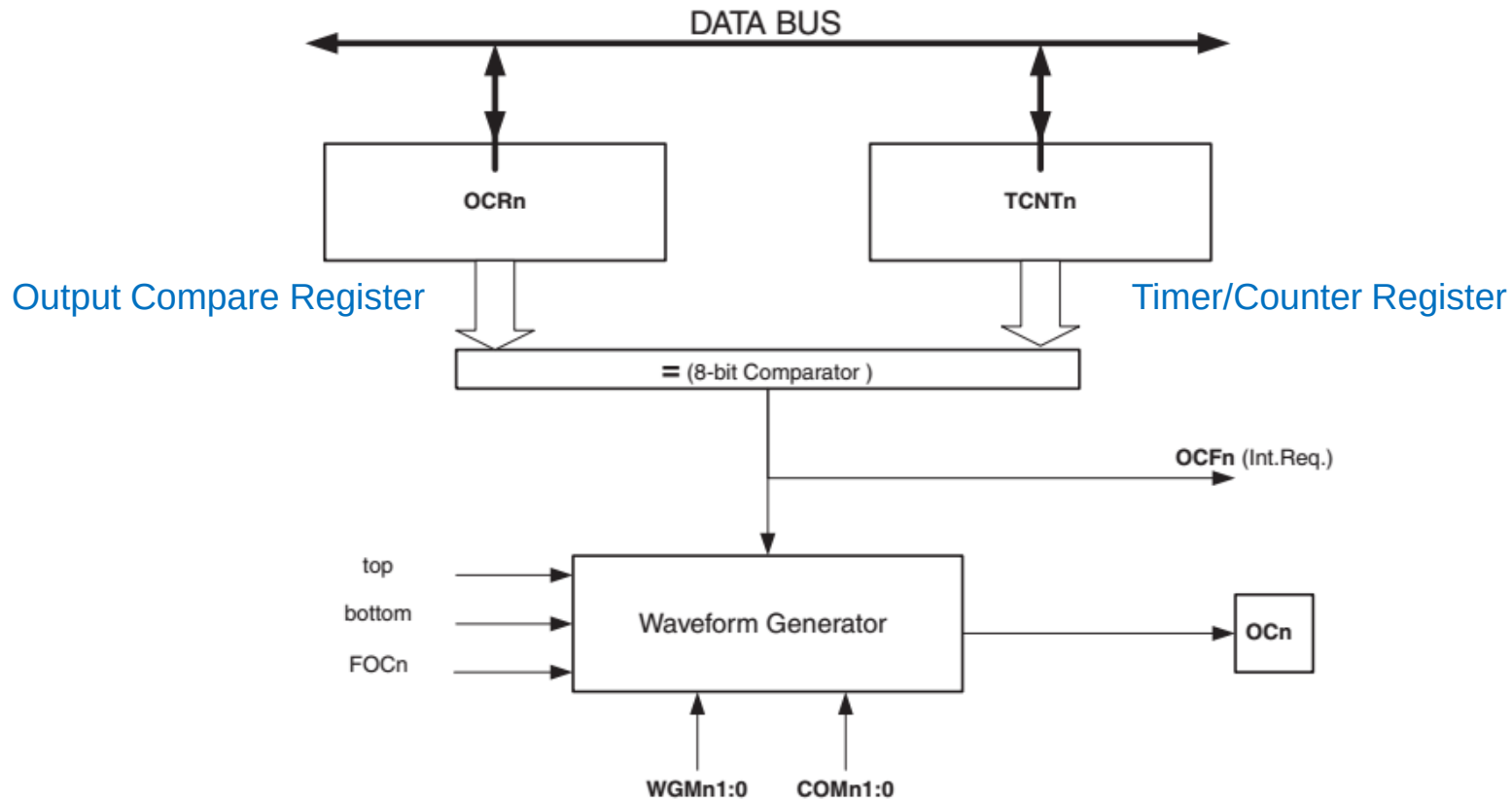
Sistema básico de temporización



- El corazón del sistema es un registro contador, llamado TCNT “Timer Counter”
- El contador suma 1 (o resta 1 según dirección de conteo) cada vez que en su entrada hay un flanco del reloj (count).
- La señal de reloj puede provenir desde el Prescaler o por terminal externo (pin Tn)
 - Si contamos pulsos del reloj del MCU tenemos una base de tiempo con prestaciones conocidas.
 - Si contamos pulsos externos de una señal “desconocida”, estamos contando cuantas veces ocurrió un flanco en el terminal Tn.
- La capacidad de conteo depende del número de bits del contador, por ejemplo con 16 bits puede contar hasta 65536 y cuando llega al máximo valor 0xFFFF se reinicia (clear) automáticamente generando un aviso de desborde u overflow (flag TOVn).
- Bottom es el valor inicial (por defecto 0)
- TOP es el valor final por lo tanto el contador es $MOD=TOP+1$

Timer con generador de señales

- El sistema se basa en comparar el valor actual del contador TCNT contra un registro que contiene un Valor almacenado (OCRn)

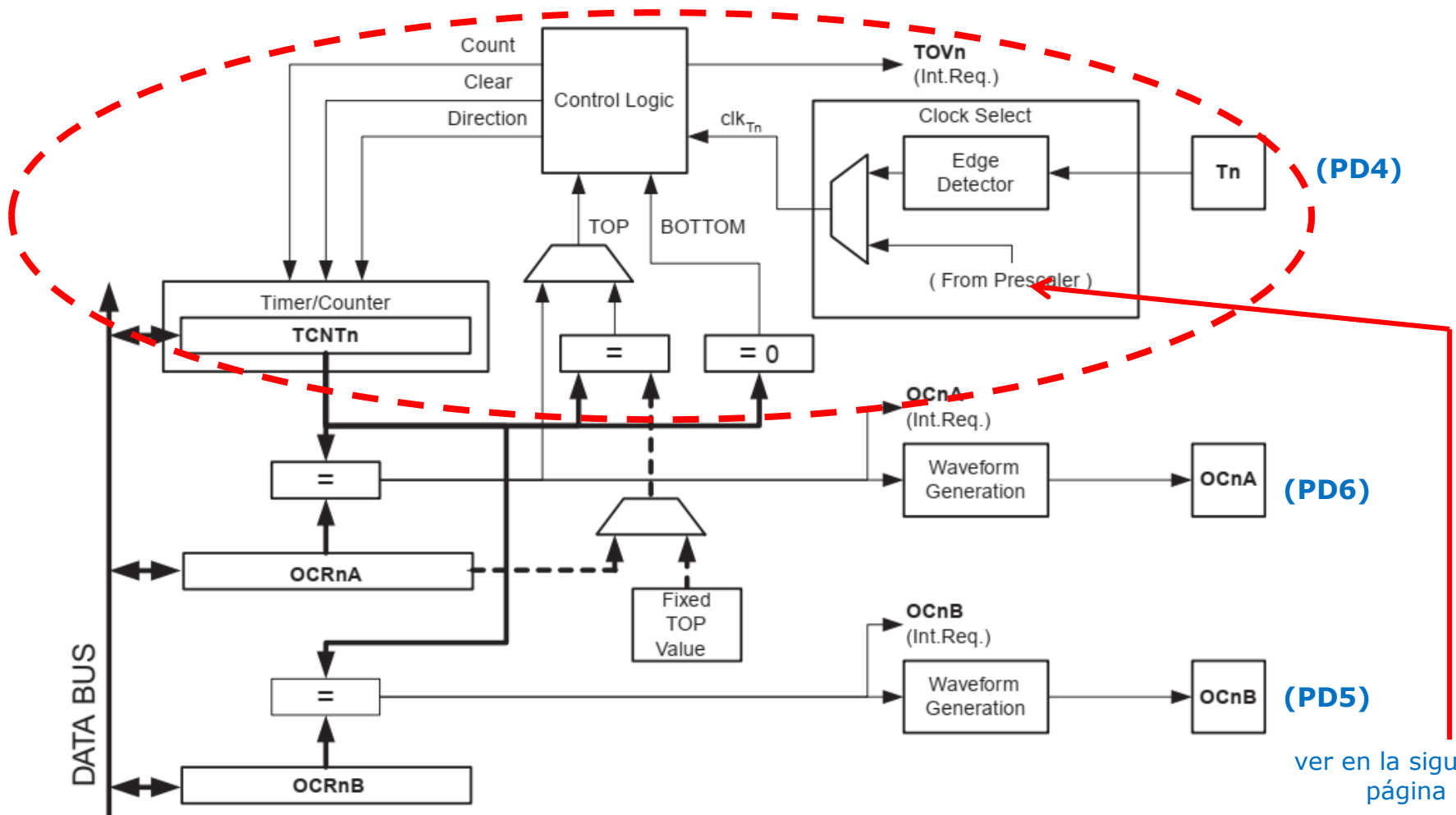


- Cuando el **TCNT** iguala al valor contenido en el registro de comparación **OCR**, se genera un aviso interno (flag de interrupción **OCF**) para que el programa tome alguna acción.
- Además, con un circuito especial (Waveform Generator) se puede modificar el estado del terminal **OCn** cuando se active el flag.
 - Aplicaciones: generación precisa de retardos, generación de señales digitales de frecuencia variable o PWM.

ATMEGA328 - TIMER 0 (8bits)

- Diagrama en bloques

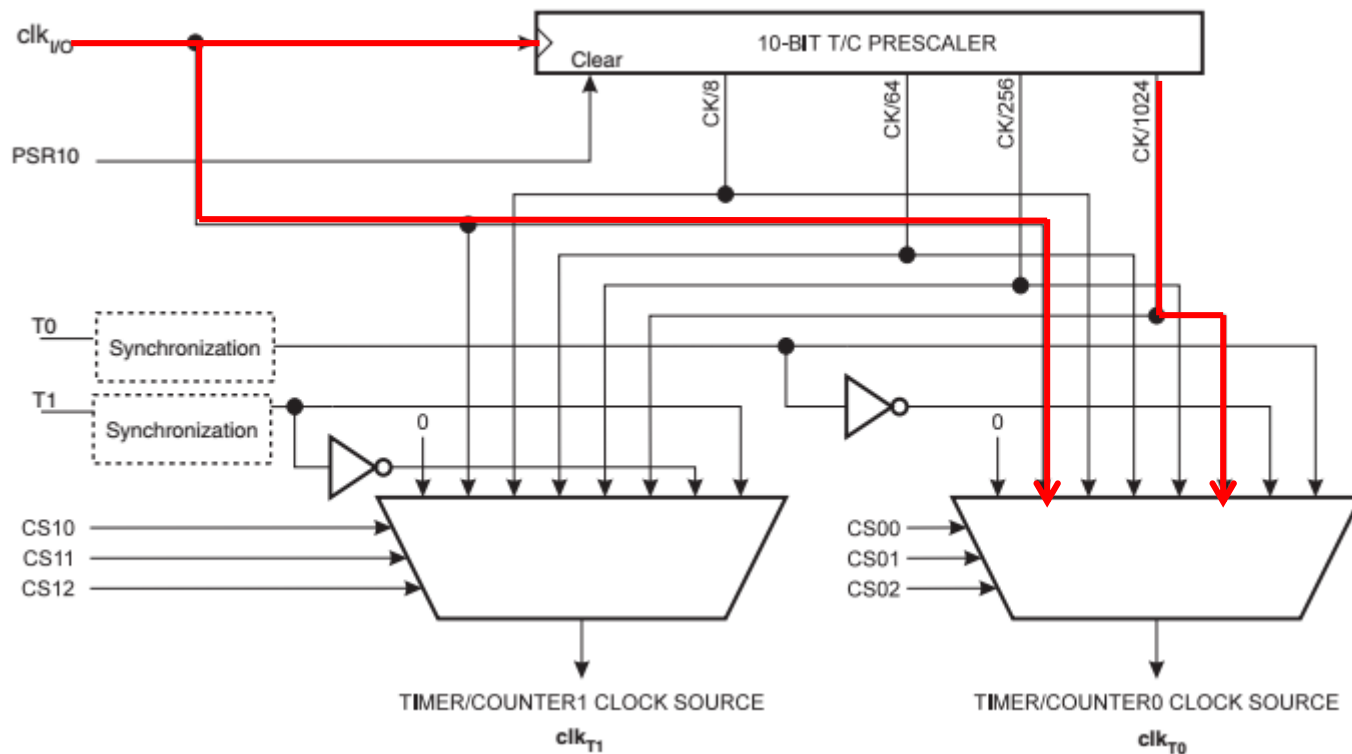
8-bit Timer/Counter0 with PWM
CH15, pg. 102, data sheet



ver en la siguiente
página

TIMER 0 -Prescalador

- TIMER 0 y TIMER 1: El PRESCALADOR permite configurar el reloj de conteo para adecuar la resolución. Además permite configurar las entradas T0, T1 para conteo de pulsos externos.

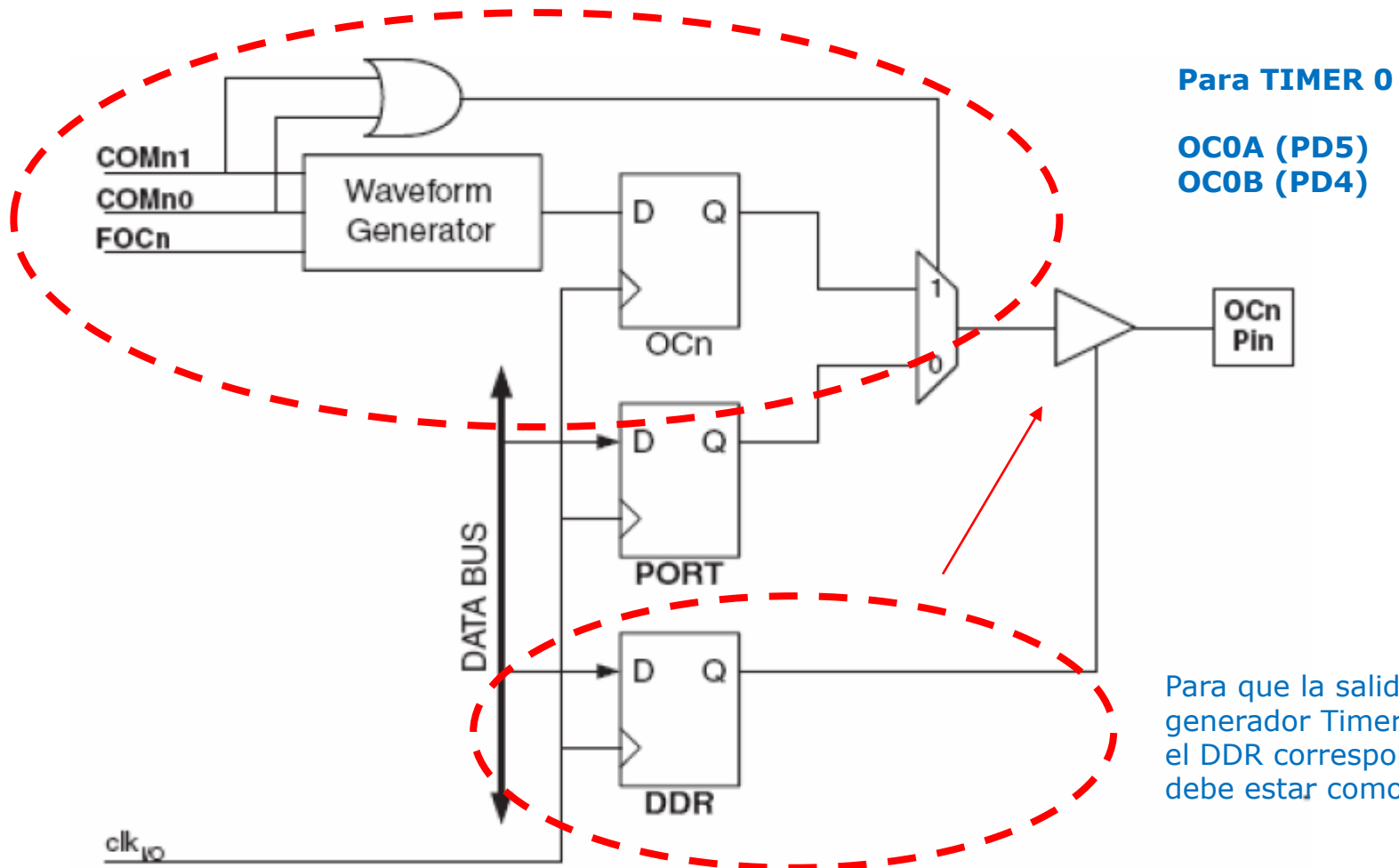


$$f_{clkT0} = f_{clkIO} / \text{PRESCALER}$$

A Timer 1

TIMER 0 – Comparador de salida

- Control de la salida de un terminal I/O



Para que la salida del generador Timer funcione el DDR correspondiente debe estar como "salida"

TIMER 0

Registros para su programación

Registros de configuración

7	6	5	4	3	2	1	0	
COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
R/W	R/W	R/W	R/W	R	R	R/W	R/W	
7	6	5	4	3	2	1	0	
FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
W	W	R	R	R/W	R/W	R/W	R/W	

Contador

7	6	5	4	3	2	1	0	
TCNT0[7:0]								TCNT0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	

Registros de comparación

7	6	5	4	3	2	1	0	
OCR0A[7:0]								OCR0A
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
7	6	5	4	3	2	1	0	
OCR0B[7:0]								OCR0B
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	

Conf. de interrupciones

7	6	5	4	3	2	1	0	
–	–	–	–	–	OCIE0B	OCIE0A	TOIE0	TIMSK0
R	R	R	R	R	R/W	R/W	R/W	

Banderas de notificación

7	6	5	4	3	2	1	0	
–	–	–	–	–	OCF0B	OCF0A	TOV0	TIFR0
R	R	R	R	R	R/W	R/W	R/W	

TIMER 0

Registros
de configuración

7	6	5	4	3	2	1	0	
COM0A1	COM0A0	COM0B1	COM0B0	–		WGM01	WGM00	TCCR0A
R/W	R/W	R/W	R/W	R	R	R/W	R/W	
7	6	5	4	3	2	1	0	
FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
W	W	R	R	R/W	R/W	R/W	R/W	

Table 14-8. Waveform Generation Mode Bit Description

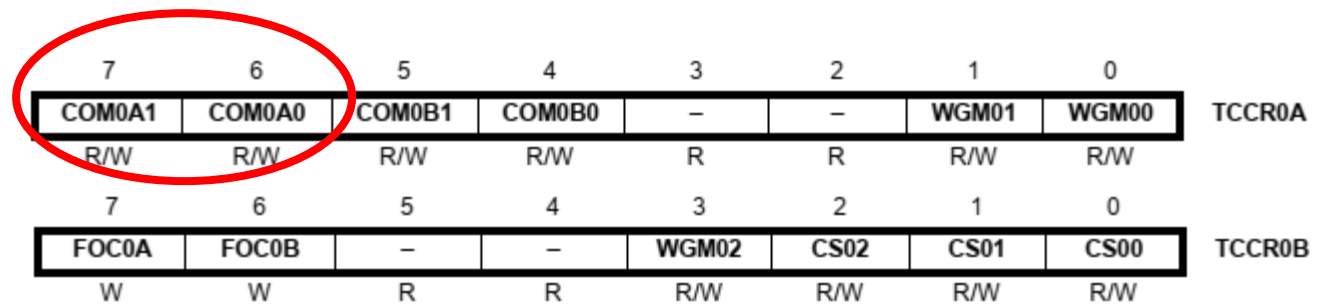
Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCR _x at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Notes: 1. MAX = 0xFF

2. BOTTOM = 0x00

CTC = Clear Timer on Compare match

TIMER 0



Recordemos que hay 2
Generadores de este tipo
Controlados por los comparadores
A y B

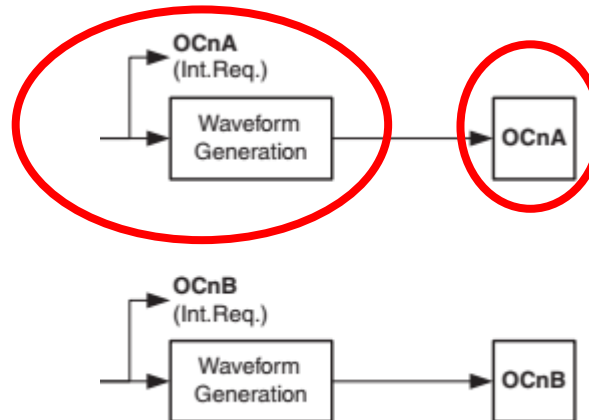


Table 14-2. Compare Output Mode, non-PWM Mode

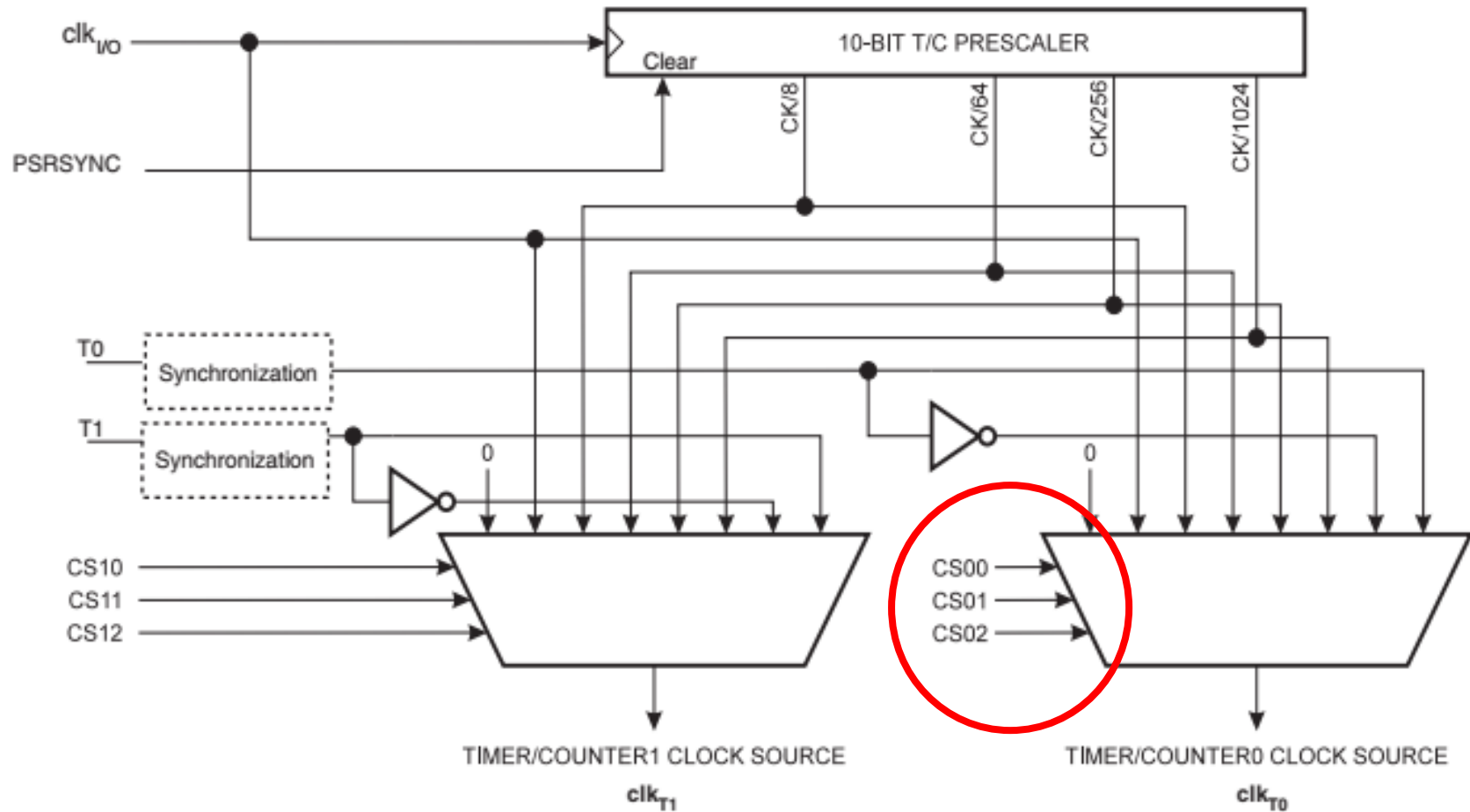
COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	Toggle OC0A on Compare Match
1	0	Clear OC0A on Compare Match
1	1	Set OC0A on Compare Match

- Hay otra tabla igual para los bits COM0Bx

No se utiliza la salida Ocnx
Si no se requiere

TIMER 0

7	6	5	4	3	2	1	0	
COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
R/W	R/W	R/W	R/W	R	R	R/W	R/W	
7	6	5	4	3	2	1	0	
FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
W	W	R	R	R/W	R/W	R/W	R/W	



TIMER 0

7	6	5	4	3	2	1	0	
COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
R/W	R/W	R/W	R/W	R	R	R/W	R/W	
7	6	5	4	3	2	1	0	
FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
W	W	R	R	R/W	R/W	R/W	R/W	

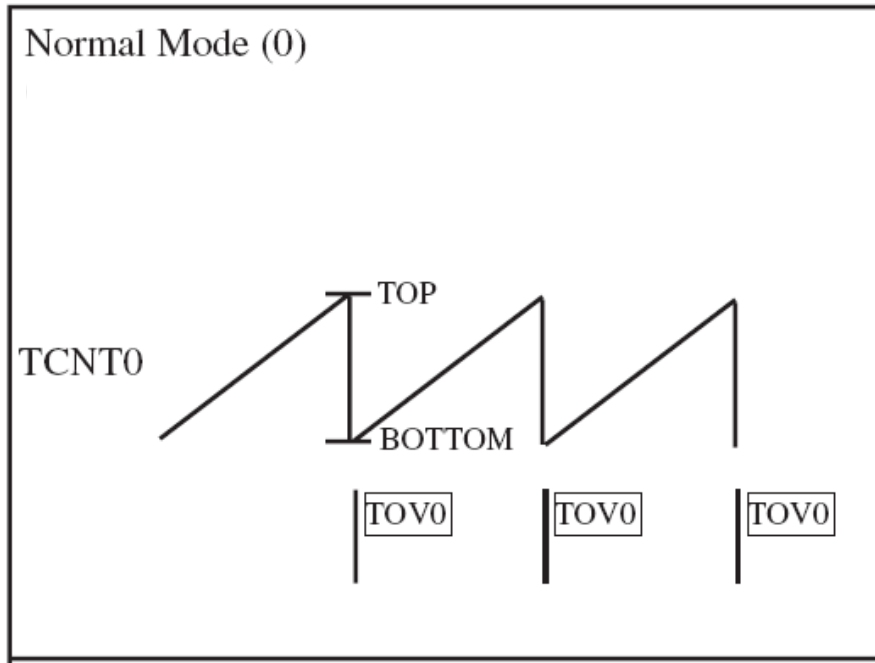
Timer detenido

Table 14-9. Clock Select Bit Description

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$ /(No prescaling)
0	1	0	$clk_{I/O}/8$ (From prescaler)
0	1	1	$clk_{I/O}/64$ (From prescaler)
1	0	0	$clk_{I/O}/256$ (From prescaler)
1	0	1	$clk_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

TIMER 0

- Modos de funcionamiento: normal



BOTTOM = 0
TOP = $2^8 - 1 = 255$

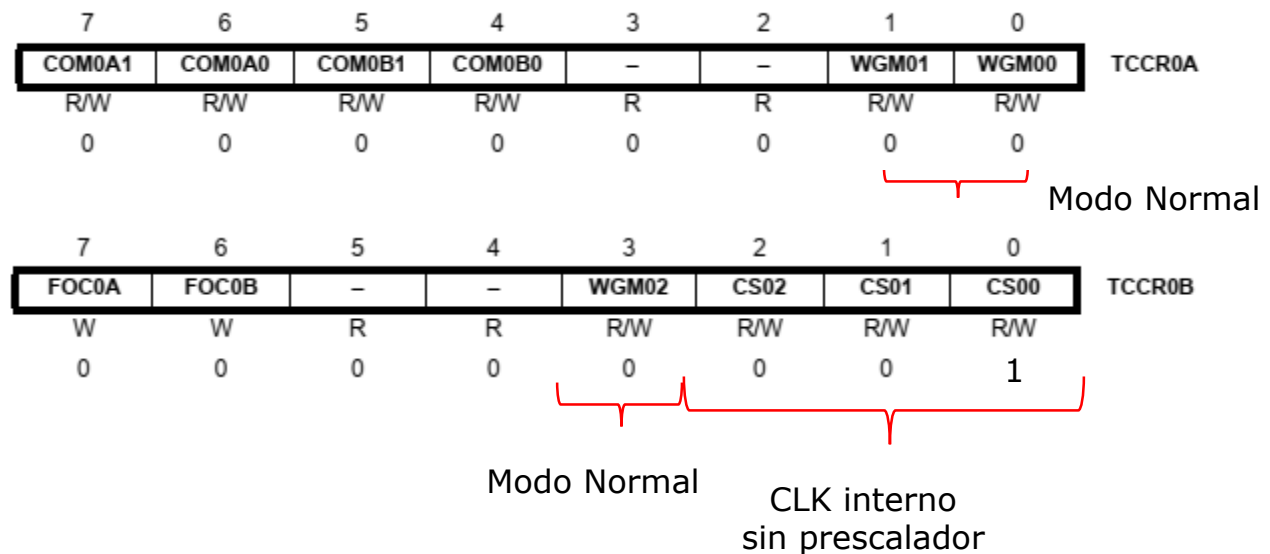
TCNT0: contador del Timer0

TOV0: flag de Overflow del Timer0

$$\Rightarrow f_{OVF} = f_{clkTn} / (TOP + 1)$$

TIMER 0

Ejemplo 1: Configurar modo normal, reloj interno sin prescalador.



- Calcular la resolución de temporización para reloj interno de 2 MHz y 16 MHz:

$$T_{clkT0} = 1 / f_{clkT0} \Rightarrow 500 \text{ ns y } 62.5 \text{ ns respectivamente}$$

- Calcular el tiempo de overflow de dicho contador:

$$T_{OVF} = 2^8 \cdot T_{clkT0} \Rightarrow 128 \text{ us y } 16 \text{ us respectivamente}$$

Se puede observar que a mejor resolución temporal $\Rightarrow$ menor es el tiempo de overflow

TIMER 0

- Modos de funcionamiento: CTC

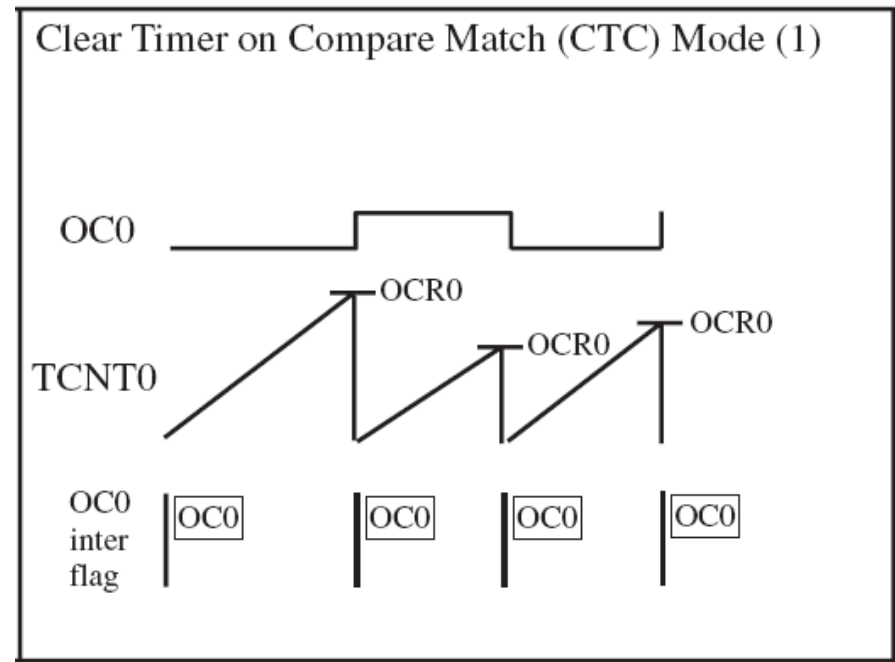
OC0x: Pin salida del periférico

OCF0x: flag de comparación

OCR0x: Registro que contiene el valor a comparar con el TCNT0

Para OCR0 fijo:

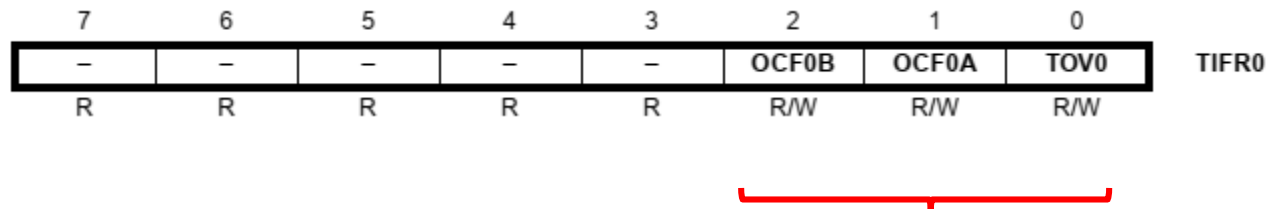
$$\Rightarrow f_{OC0x} = f_{clkTn} / (OCR0x + 1)$$



Esta es la base para generar una interrupción periódica o base de tiempo programable como vimos al principio de la clase

TIMER 0

- Flags o banderas de condición del Timer 0 en el Registro **TIFR0**:



- Activación de los flags:

TOV0 =1 cuando el contador pasa de FF a 00 (desborde u overflow)

OCF0x =1 cuando el contador iguala al valor del registro OCR0x

- Borrado de los flags:

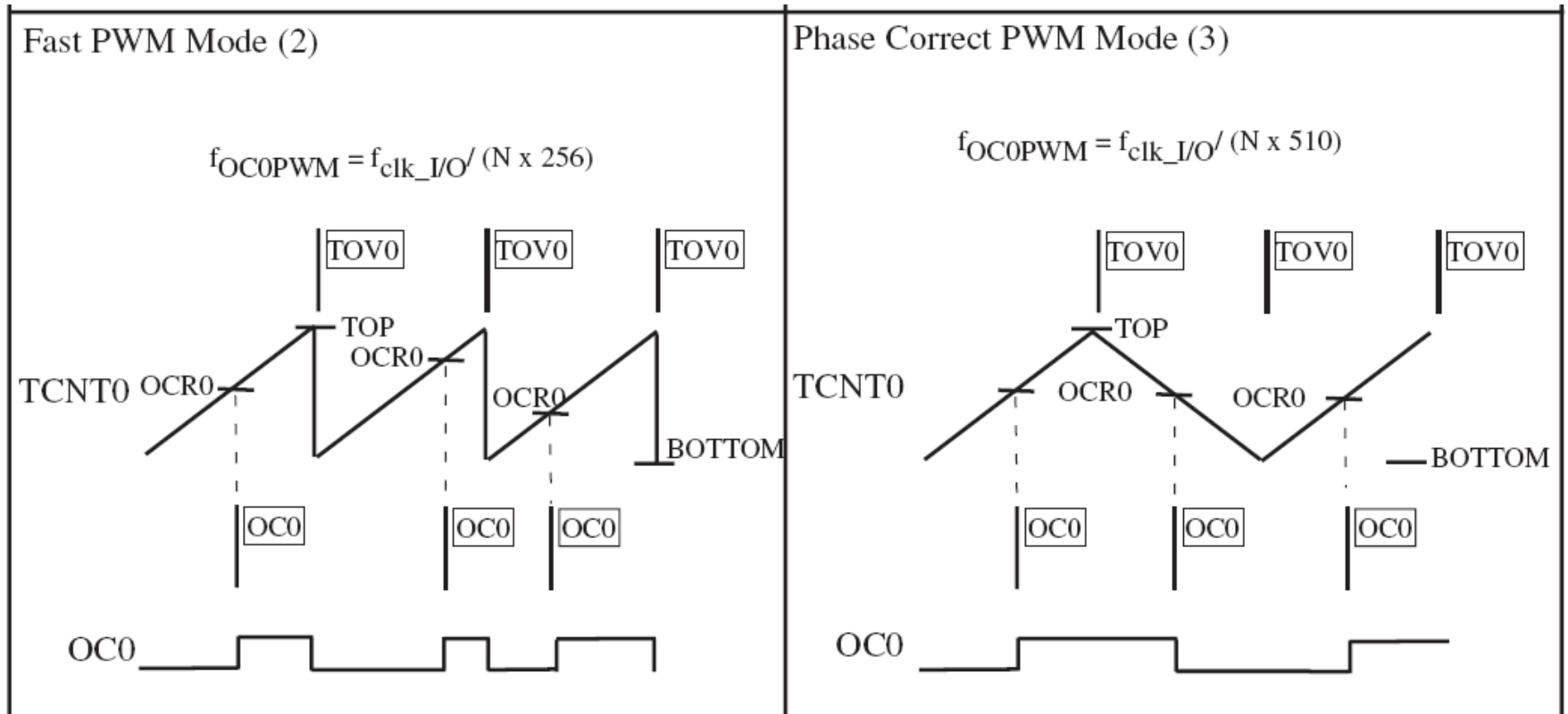
- OCF0x se borra automáticamente cuando se ejecuta el servicio de interrupción correspondiente.
- En modo polling el programador debe volver a 0 los flags escribiendo un "1" en dichos flags

```
TIFR0 |= (1<<TOV0) ;
```

```
TIFR0 |= (1<<OCF0A) ;
```

TIMER 0

- **Modos de funcionamiento: PWM**



Los veremos en el TP4...

TIMER 0

- Ejemplo 2: invertir PB4 cada 70useg utilizando un retardo con el TIMER 0

```
#include "avr/io.h"

void T0Delay(void) ;

int main(void) {

    DDRB |= (1<<PORTB4) ;
    while(1) {
        T0Delay() ;
        PORTB^= (1<<PORTB4) ;
    }
}
```

TIMER 0

- Ejemplo 2: invertir PB4 cada 70useg utilizando un retardo con el TIMER 0

```
//70us=>70 ciclos de 1us  
//Si fclk=8MHz => ftimer0=fclk/prescaler=8/8=1MHz  
//Ttimer0=1us  
//70 ciclos => 256-70=186
```

Notar que supuse fclk=8MHz

```
void T0Delay(void) {  
    TCNT0=186;  
    TCCR0A=0x00;  
    TCCR0B=0x02; //modo normal, prescaler 8  
    while (TIFR0 & (1<<TOV0) == 0);  
    TCCR0B=0x00; //apagar timer0  
    TIFR0 |= (1<<TOV0);  
}
```

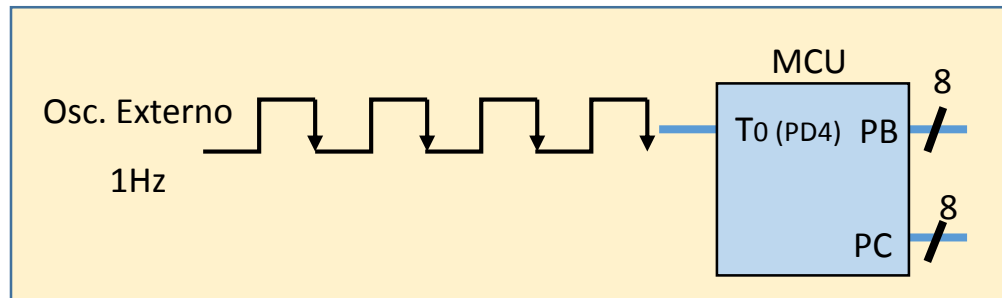
Algoritmo básico para generar un retardo (delay)

- 1) Cargar el TCNT0 con el valor inicial de conteo (el retardo será TOF-inicial)
- 2) Cargar en el TCCR0 la configuración deseada. Al elegir la fuente de reloj (el timer comienza el conteo)
- 3) Verificar mediante el flag de Tov0 si se alcanzó el valor FF en el contador
- 4) Detener el contador desconectando la fuente de reloj
- 5) borrar el flag Tov0

TIMER 0

- Ejemplo 3: Utilización como contador de eventos externos

Supongamos que disponemos de un reloj externo de 1Hz conectado al pin T0



Vamos a visualizar en los puertos de salida el conteo de pulsos (16bits) cada 1seg.



TIMER 0

- Ejemplo 3: Utilización como contador de eventos externos

```
#include "avr/io.h"

int main(void) {

    DDRD&=~(1<<PORTD4);
    PORTD|=(1<<PORTD4);
    DDRB=DDRC=0xFF;
    TCCR0A=0x00;
    TCCR0B=0x06; //external clock T0 pin
    TCNT0=0;

    while(1) {
        do{
            PORTB=TCNT0;
        }while (TIFR0&(1<<TOV0)==0);
        TIFR0|=(1<<TOV0);
        PORTC++;
    }
}
```

Notar que en cada overflow sumo 1 al PORTC, extendiendo la capacidad de conteo del TCNT0

TIMER 0

- Ejemplo 4: invertir PB5 utilizando interrupción de comparador A del TIMER 0

```
#include "avr/io.h"
#include "avr/interrupt.h"

int main(void) {

    DDRB |= (1<<PORTB5) ;

    TCCR0A=0x02;    //modo CTC
    TCCR0B=0x01;    //clk prescaler=1
    OCR0A=40;       //módulo
    TIMSK0=(1<<OCIE0A); //int comp A
    sei() ;

    while(1){
        //Tarea de Background
    }

ISR(TIMERO0_COMPA_vect)
{
    PORTB^=(1<<PORTB5) ;
}
```

Interrupción periódica del comparador A

¿preguntas?

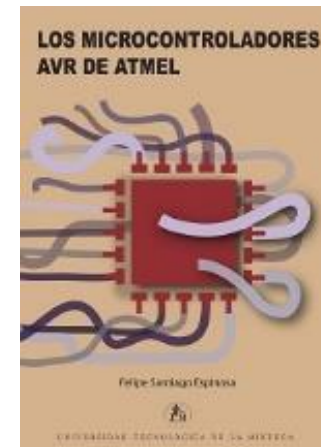
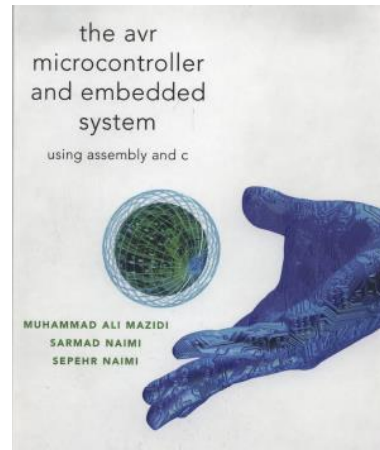
Tarea:

- Graficar la señal de salida en PB5 en función del tiempo
- Calcular la frecuencia de la señal de salida en PB5, asumiendo $f_{clk}=16\text{MHz}$
- Simular y verificar el comportamiento en Proteus con un Osciloscopio
- ¿afecta a la señal generada en PB5 la latencia de la interrupción?

- ¿puedo usar el timer para generar una interrupción periódica de 10ms e implementar un reloj con horas, min y seg (HH:MM:SS)?
- ¿y si hay otras interrupciones habilitadas (y de mayor prioridad) será preciso mi reloj?

Bibliografía:

- *The AVR microcontroller & Embedded Systems*. Mazidi, Naimis, CH9



- Libro Digital de Felipe Espinosa, CH4

- Hojas de datos Atmega 328p, Atmega2560, otros AVR.

- tutorial: [Timer arduino](#)

